⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Clause: 4 — General Requirements · **Register:** Document Register

What the standard requires

REF	TITLE	APPLIES AT CLASS C	WHAT MUST BE PRODUCED
4.1	Quality management system	Yes	See "seeAlso" note below — 4.1 has no `output` of its own; it is satisfied by pointing at a QMS.
4.2	Risk management	Yes	See "seeAlso" note below — satisfied by pointing at ISO 14971.
4.3	Software safety classification	Yes	"Software safety class of each software system and software item, recorded in the risk management file, with a rationale where an item is classified differently from its parent."
4.4	Legacy software	Yes	"Gap analysis against 5.2, 5.3, 5.7 and Clause 7; a plan to close the gaps, with software system test records as the minimum deliverable; and the legacy software version plus a documented rationale for its continued use."

(Quoted directly from `applicability.json` — Clause 4 carries no per-requirement `[Class ...]` tags in the standard itself; Table A.1 assigns all four sub-clauses to every class.)

4.1 — Quality management system

The requirement is to demonstrate the ability to consistently meet customer and applicable regulatory requirements, which the standard says can be shown by an ISO 13485 QMS, a national QMS standard, or a QMS required by national regulation (full text in `applicability.json`, `general-requirements → 4.1`).

How this project actually works: there is no certified QMS — this is a single-maintainer repository, not a manufacturer. What stands in for it:

- A written record of *why* a decision was made, not just what it was — the project's design notes and this document set follow that discipline throughout, which is closer in spirit to a QMS's document-control requirement than a certificate would be on its own.
- Every change to regulatory content (the safety-class mapping in `applicability.json`) is reviewed against the standard's normative text before being accepted — see 11 — Risk Management File.
- An automated test suite acts as the change-acceptance gate a QMS would otherwise mandate manually: `tests/test_site.py` runs before anything is considered finished.

Gap: none of this is certified or externally audited. That is a real and permanent difference from a manufacturer's QMS, stated here rather than implied away.

4.2 — Risk management

The normative text requires a risk management process complying with ISO 14971, named as the single route with no alternative offered (unlike 4.1's three routes). See 11 — Software Risk Management File for how this project applies that idea to itself, reflexively: not to patient risk, since this is not a medical device, but to the risk of teaching a reader something regulatorily wrong.

4.3 — Software safety classification

This is the one sub-clause of Clause 4 worth actually working through, rather than cross-referencing, because the reasoning is the point.

The test, as Amendment 1 states it (from `data/phases.json , general-requirements`): *"A software item that could contribute to a hazardous situation must be Class B minimum; if it is the sole control measure preventing serious injury or death, it must be Class C. Software items that cannot contribute to any hazardous situation may be Class A."*

Applying it to this repository as it actually stands: the training site cannot contribute to a hazardous situation in the sense §4.3 means — it does not control, monitor, or inform the treatment of a patient. It is content about a standard, not an implementation of one. A literal classification puts it outside IEC 62304's scope altogether: **no class applies, because the standard does not apply.**

Why this document set proceeds at Class C anyway: the document register records that Class C was chosen as a **teaching target** for this demonstration set — the largest, most complete version of the lifecycle document list, which is more useful to a learner than the true (and much shorter) answer of "not applicable." Every document in this set says so where it matters, and none of them should be read as an actual classification record for a real device.

Gap: a real risk management file would record this reasoning as "rationale where an item is classified differently from its parent" per the `output` text above. This section *is* that rationale, kept here rather than duplicated in 11.

4.4 — Legacy software

Amendment 1's legacy provisions exist for software already on the market without sufficient evidence it was developed to the current standard. This project has no such history — it has been developed from a single lifecycle model throughout, recorded in `git log` and 12 — Configuration Management Plan. **Not applicable, and recorded as such rather than left blank**, which is the same

principle the site's own deliverables list uses for sub-clauses with no `output`: an absence is shown, not silently omitted.